



# REACT

## INTERVIEW PREPARATION BOOKLET

From Basics to Advanced • Code Snippets Included • All Difficulty Levels

- Easy Questions
- Medium Questions
- Hard Questions
- Expert-Level Questions
- Live Coding Challenges
- System Design & Architecture



## HOW TO USE THIS BOOKLET

This booklet is your one-stop resource for cracking any React interview — from junior to staff-level. Every question is tagged by difficulty so you can prioritize based on the role you're targeting.

### Difficulty Legend

[Easy] — Fresher to 1 yr | [Medium] — 1-3 yrs | [Hard] — 3-5 yrs | [Expert] — 5+ yrs / Senior roles

## Chapters at a Glance

- 1. React Fundamentals
- 2. JSX & Rendering
- 3. Components & Props
- 4. State & Lifecycle
- 5. React Hooks — Deep Dive
- 6. Event Handling & Forms
- 7. Lists, Keys & Conditional Rendering
- 8. Performance Optimization
- 9. Context API & State Management
- 10. React Router
- 11. Error Boundaries & Suspense
- 12. Higher-Order Components & Render Props
- 13. Custom Hooks
- 14. Testing in React
- 15. Live Coding Challenges
- 16. Advanced & Expert Questions

## CHAPTER 1 · React Fundamentals

### Core Concepts You Must Know

#### **What is React?**

React is an open-source JavaScript library (not a framework) built by Facebook for building fast, interactive user interfaces. It uses a component-based architecture and a Virtual DOM to efficiently update the UI.

#### **[Easy] What is React and why would you use it?**

React is a UI library that lets you build reusable components. You use it because: (1) Virtual DOM makes updates fast, (2) Component reusability saves time, (3) Unidirectional data flow makes debugging easier, (4) Huge ecosystem and community support.

#### **[Easy] What is the Virtual DOM? How is it different from the real DOM?**

The Virtual DOM is a lightweight JavaScript copy of the real DOM kept in memory. When state changes, React creates a new Virtual DOM tree, compares it with the previous one (diffing), and only updates the parts of the real DOM that actually changed (reconciliation). This is much faster than updating the entire real DOM.

#### **[Medium] What is Reconciliation in React?**

Reconciliation is the process React uses to update the DOM. When a component's state/props change, React re-renders and creates a new Virtual DOM tree. It then uses a diffing algorithm ( $O(n)$  complexity) to compare it with the old tree and applies only the minimal set of changes to the real DOM.

#### **[Easy] What is the difference between React and React DOM?**

'react' package provides the core library (components, hooks, createElement). 'react-dom' is the renderer that connects React to the actual browser DOM. This separation allows React to target different environments (web, native, server) with the same core.

#### **[Hard] What is the React Fiber architecture?**

React Fiber (React 16+) is a complete rewrite of React's reconciliation algorithm. It breaks rendering work into small units (fibers) that can be paused, resumed, and prioritized. This enables features like Concurrent Mode, Suspense, and time-slicing. Key benefit: high-priority updates (like user input) can interrupt low-priority work (like fetching data).

---

### One-Liner Quick-Fire Q&A

**Is React a framework or a library?**

A library — it handles only the View layer. You pick routing, state management, etc. yourself.

**What is Babel and why is it needed with React?**

Babel is a JS transpiler that converts JSX and modern ES6+ code into plain JavaScript that browsers understand.

**What is Webpack in context of React?**

A module bundler that compiles all JS, CSS, images into bundles optimized for the browser.

**What is Create React App (CRA)?**

A CLI tool that sets up a zero-config React project with Webpack and Babel pre-configured.

**What is Vite?**

A next-gen build tool (alternative to CRA) that uses native ES modules for lightning-fast dev server startup.

## CHAPTER 2 · JSX & Rendering

### [Easy] What is JSX?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup inside JavaScript. It is NOT HTML — it compiles down to `React.createElement()` calls. It must have a single root element, use `className` instead of `class`, and close all tags.

### JSX compiles to:

```
// JSX you write:
const element = <h1 className="title">Hello, World!</h1>;

// What Babel compiles it to:
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello, World!'
);
```

### [Easy] What are the rules of JSX?

1. Must return a single root element (use `<> </>` Fragment if needed). 2. All tags must be closed (`<br />` not `<br>`). 3. Use camelCase for attributes (`className`, `onClick`, `htmlFor`). 4. JavaScript expressions go inside curly braces `{}`. 5. Comments inside JSX: `{/* comment */}`.

### JSX Expression Examples:

```
const name = 'Rohan';
const isLoggedIn = true;

// Embedding expressions
const greeting = <h1>Hello, {name}!</h1>;

// Inline conditional rendering
const ui = <div>{isLoggedIn ? <Dashboard /> : <Login />}</div>;

// Short-circuit evaluation
const msg = <div>{isLoggedIn && <p>Welcome back!</p>}</div>;

// Fragments — no extra DOM node
return (
  <>
    <Header />
    <Main />
  </>
);
```

### [Medium] Can you write React without JSX?

Yes! JSX is just syntactic sugar. You can use `React.createElement(type, props, ...children)` directly. But JSX is much more readable for complex trees.

## CHAPTER 3 · Components & Props

### [Easy] What is the difference between functional and class components?

Functional components are plain JavaScript functions that return JSX. Class components extend `React.Component` and must have a `render()` method. Since React 16.8, hooks allow functional components to use state and lifecycle — functional components are now preferred.

```
// Functional Component (modern, preferred)
function Welcome({ name }) {
  return <h1>Hello, {name}!</h1>;
}

// Class Component (legacy)
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

### [Easy] What are props? Are they mutable?

Props (properties) are read-only inputs passed from parent to child component. They are immutable — a component must never modify its own props. If you need to change data, use state. This enforces one-way (unidirectional) data flow.

### [Medium] What is prop drilling and how do you avoid it?

Prop drilling is passing props through many component layers even though intermediate components don't need them. Avoid it with: (1) React Context API, (2) State management libraries like Redux/Zustand, (3) Component composition patterns.

### [Easy] What are default props?

Default props provide fallback values when a prop is not passed. In functional components use default parameter values. In class components use `ComponentName.defaultProps = {...}`.

```
// Functional — default parameters
function Button({ label = 'Click Me', type = 'button' }) {
  return <button type={type}>{label}</button>;
}

// Class — defaultProps
class Button extends React.Component { ... }
Button.defaultProps = { label: 'Click Me', type: 'button' };
```

### [Easy] What is PropTypes and why use it?

PropTypes is a runtime type-checking library. You attach a .propTypes object to your component describing expected prop types. React logs warnings in development if types don't match. For production apps TypeScript is preferred over PropTypes.

```
import PropTypes from 'prop-types';

function UserCard({ name, age, isAdmin }) { ... }

UserCard.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
  isAdmin: PropTypes.bool,
  onClick: PropTypes.func,
};
```

### [Easy] What is children prop?

children is a special prop that represents whatever is between opening and closing tags of your component. It allows component composition — you can wrap any content inside a component.

```
function Card({ children, title }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-body">{children}</div>
    </div>
  );
}

// Usage:
<Card title="Profile">
  <p>Name: Rohan</p>
  <p>Age: 25</p>
</Card>
```



## CHAPTER 4 · State & Lifecycle

### [Easy] What is state in React?

State is a built-in object in components that stores dynamic data that can change over time. When state changes, React re-renders the component. Unlike props, state is managed within the component (local state).

### [Easy] What is the difference between props and state?

Props are passed from parent to child (external, read-only). State is managed within the component itself (internal, mutable via `setState/useState`). Props are like function arguments; state is like local variables.

### [Medium] Why should you never mutate state directly?

Directly mutating state (e.g., `this.state.count = 5`) does not trigger a re-render. React needs `setState()/useState` setter to know something changed so it can schedule a re-render. Direct mutation also breaks `PureComponent/React.memo` optimizations.

```
// ❌ WRONG — no re-render triggered
this.state.items.push(newItem);

// ✅ CORRECT — always create a new reference
this.setState({ items: [...this.state.items, newItem] });

// ✅ With useState hook:
setItems(prev => [...prev, newItem]);
```

## Class Component Lifecycle Methods

### The Three Phases

Mounting → Updating → Unmounting

```
class MyComponent extends React.Component {
  // MOUNTING
  constructor(props) {
    super(props);
    this.state = { count: 0 }; // Initialize state
  }

  static getDerivedStateFromProps(props, state) {
    return null; // Rarely used
  }

  componentDidMount() { // After first render — good for API calls
    fetchData().then(data => this.setState({ data }));
  }
}
```

```

// UPDATING
shouldComponentUpdate(nextProps, nextState) {
  return nextState.count !== this.state.count; // Optimize re-renders
}

getSnapshotBeforeUpdate(prevProps, prevState) {
  return null; // Capture DOM info before update
}

componentDidUpdate(prevProps, prevState, snapshot) {
  if (prevState.count !== this.state.count) { // Guard against infinite loop!
    console.log('Count changed');
  }
}

// UNMOUNTING
componentWillUnmount() { // Cleanup: cancel timers, unsubscribe
  clearInterval(this.timer);
}

render() {
  return <div>{this.state.count}</div>;
}
}

```

### **[Medium] What is setState and is it synchronous or asynchronous?**

setState() in class components is asynchronous (batched). React batches multiple setState calls in event handlers for performance. If you need the value immediately after setting, use the callback form: setState(updater, callback). With React 18, automatic batching extends to all async contexts too.

```

// ❌ Might use stale state
this.setState({ count: this.state.count + 1 });
this.setState({ count: this.state.count + 1 }); // Both read same old value!

// ✅ Use functional updater to get latest state
this.setState(prev => ({ count: prev.count + 1 }));
this.setState(prev => ({ count: prev.count + 1 })); // Correctly adds 2

```

## CHAPTER 5 · React Hooks — Deep Dive

### Rules of Hooks (CRITICAL — examiners love this)

1. Only call hooks at the TOP LEVEL — never inside loops, conditions, or nested functions. 2. Only call hooks from React FUNCTION COMPONENTS or custom hooks. These rules exist because React relies on hook call order to associate state with each hook.

## useState

### [Easy] Explain useState hook with an example

useState returns a stateful value and a setter function. The setter accepts either a new value or a function that receives the previous state (functional update). Initial state is only used on first render.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  // Direct update
  const increment = () => setCount(count + 1);

  // Functional update (safer for derived state)
  const incrementSafe = () => setCount(prev => prev + 1);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

### [Medium] What is lazy initialization in useState?

If computing the initial state is expensive, pass a function instead of a value. React calls it only on the first render. Example: `useState(() => expensiveComputation())` vs `useState(expensiveComputation())` — the latter runs on every render!

## useEffect

### [Easy] Explain useEffect and its dependency array

useEffect runs a side effect after every render. The dependency array controls when it re-runs: `[]` means run once (on mount), `[dep1, dep2]` means run when those values change, no array means run after every render. The cleanup function (returned from `useEffect`) runs before the next effect and on unmount.

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let isCancelled = false;          // Prevent state update on unmounted
    component

    async function fetchUser() {
      const data = await fetch(`/api/users/${userId}`).then(r => r.json());
      if (!isCancelled) setUser(data);
    }

    fetchUser();

    return () => {                    // Cleanup function
      isCancelled = true;
    };
  }, [userId]);                      // Re-run whenever userId changes

  return user ? <div>{user.name}</div> : <p>Loading...</p>;
}
```

#### **[Medium] What are common mistakes with useEffect?**

1. Missing dependencies (stale closure bug — you read an old value). 2. Not cleaning up subscriptions/timers (memory leaks). 3. Putting async function directly as useEffect callback (it returns a Promise, not a cleanup function — define async function inside and call it). 4. Infinite loop by adding an object/array to deps without memoizing it.

#### **[Hard] How do you prevent infinite loops in useEffect?**

An infinite loop happens when your effect updates something that is also in the dependency array. Fix: (1) Ensure the update is conditional, (2) Use useCallback/useMemo to stabilize function/object references in deps, (3) Separate effects for different concerns.

## useContext

#### **[Medium] How does useContext work?**

useContext lets you read a context value inside a component without prop drilling. You first create a context with createContext(defaultValue), wrap a subtree with <Context.Provider value={...}>, then any descendant can call useContext(Context) to read the value.

```
// 1. Create context
const ThemeContext = React.createContext('light');

// 2. Provide it
function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}
```

```
// 3. Consume it (anywhere in the tree)
function Toolbar() {
  const { theme, setTheme } = useContext(ThemeContext);
  return <button onClick={() => setTheme('dark')}>Theme: {theme}</button>;
}
```

## useRef

### [Medium] What is useRef and when would you use it?

useRef returns a mutable object { current: value } that persists across renders. Updating ref.current does NOT trigger re-render. Use cases: (1) Accessing/manipulating DOM elements directly, (2) Storing previous values, (3) Storing mutable values like timer IDs without causing re-renders.

```
// Use case 1: DOM access
function TextInput() {
  const inputRef = useRef(null);
  const focus = () => inputRef.current.focus();
  return <><input ref={inputRef} /><button onClick={focus}>Focus</button></>;
}

// Use case 2: Store previous value
function Counter() {
  const [count, setCount] = useState(0);
  const prevCount = useRef(count);

  useEffect(() => {
    prevCount.current = count; // Update after render
  });

  return <p>Now: {count}, Before: {prevCount.current}</p>;
}

// Use case 3: Timer ID
function Timer() {
  const timerRef = useRef(null);
  const start = () => { timerRef.current = setInterval(() => {}, 1000); };
  const stop = () => clearInterval(timerRef.current);
  return <><button onClick={start}>Start</button><button
onClick={stop}>Stop</button></>;
}
```

## useMemo & useCallback

### [Medium] What is the difference between useMemo and useCallback?

useMemo memoizes the RESULT of a computation — returns a cached value. useCallback memoizes a FUNCTION itself — returns a stable function reference. Both only recompute when their dependency array changes. Analogy: useMemo = memo(calculate()), useCallback = memo(() => {})

```
// useMemo - cache expensive computed value
const sortedList = useMemo(() => {
  return [...items].sort((a, b) => a.name.localeCompare(b.name));
});
```

```

}, [items]); // Only re-sorts when 'items' changes

// useCallback - stable function reference (avoids child re-render)
const handleClick = useCallback((id) => {
  dispatch({ type: 'SELECT', id });
}, [dispatch]); // Stable as long as dispatch doesn't change

// When to use:
// useMemo - expensive calculations, derived data for render
// useCallback - passing callbacks to memoized child components

```

**[Hard] Does wrapping everything in useMemo/useCallback improve performance?**

NO! Memoization itself has a cost (closure creation, cache comparison). Overusing it can hurt performance. Use them only when: (1) a computation is measurably expensive, (2) a function is passed to React.memo child or used in useEffect deps, (3) you've profiled and confirmed the bottleneck.

## useReducer

**[Medium] When would you use useReducer over useState?**

Use useReducer when: (1) state has complex shape or multiple sub-values, (2) next state depends on the previous one, (3) you have multiple related actions. It's like a mini Redux inside a component. Signature: `const [state, dispatch] = useReducer(reducer, initialState)`

```

const initialState = { count: 0, error: null };

function reducer(state, action) {
  switch (action.type) {
    case 'INCREMENT': return { ...state, count: state.count + 1 };
    case 'DECREMENT': return { ...state, count: state.count - 1 };
    case 'RESET':      return initialState;
    case 'SET_ERROR':  return { ...state, error: action.payload };
    default: throw new Error(`Unknown action: ${action.type}`);
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'INCREMENT' })}></button>
      <button onClick={() => dispatch({ type: 'DECREMENT' })}></button>
      <button onClick={() => dispatch({ type: 'RESET' })}>Reset</button>
    </div>
  );
}

```

## CHAPTER 6 · Event Handling & Forms

### [Easy] How does event handling work in React?

React uses Synthetic Events — wrappers around native browser events that normalize behavior across browsers. Event handlers are camelCase (onClick, onChange, onSubmit). Event delegation is used: React attaches a single listener at the root, not on each element.

```
// Prevent default + stop propagation
function Link() {
  function handleClick(e) {
    e.preventDefault(); // Stop browser default
    e.stopPropagation(); // Stop event bubbling
    console.log('Clicked!');
  }
  return <a href="/" onClick={handleClick}>Click me</a>;
}
```

### [Medium] What is a controlled vs uncontrolled component?

Controlled: form element value is driven by React state (onChange + state). Single source of truth. Uncontrolled: form element manages its own state — you access values via refs (useRef). Controlled components are preferred because they make validation and conditional disabling easy.

```
// ✅ Controlled Component
function ControlledInput() {
  const [value, setValue] = useState('');
  return (
    <input
      value={value}
      onChange={e => setValue(e.target.value)}
      placeholder="Type here..."
    />
  );
}

// Uncontrolled Component
function UncontrolledInput() {
  const inputRef = useRef();
  const handleSubmit = () => alert(inputRef.current.value);
  return <><input ref={inputRef} /><button
    onClick={handleSubmit}>Submit</button></>;
}
```

### [Medium] Write a controlled form with validation

Key pattern: store values and errors in state, validate onChange or onBlur, disable submit if errors exist.

```
function LoginForm() {
  const [form, setForm] = useState({ email: '', password: '' });
  const [errors, setErrors] = useState({});
```

```

const validate = (name, value) => {
  if (name === 'email' && !/^\S+@\S+\.\S+/.test(value))
    return 'Invalid email';
  if (name === 'password' && value.length < 6)
    return 'Min 6 characters';
  return '';
};

const handleChange = (e) => {
  const { name, value } = e.target;
  setForm(prev => ({ ...prev, [name]: value }));
  setErrors(prev => ({ ...prev, [name]: validate(name, value) }));
};

const handleSubmit = (e) => {
  e.preventDefault();
  console.log('Submit:', form);
};

const isValid = !errors.email && !errors.password && form.email && form.password;

return (
  <form onSubmit={handleSubmit}>
    <input name="email" value={form.email} onChange={handleChange} />
    {errors.email && <span>{errors.email}</span>}
    <input name="password" type="password" value={form.password}
onChange={handleChange} />
    {errors.password && <span>{errors.password}</span>}
    <button type="submit" disabled={!isValid}>Login</button>
  </form>
);
}

```



## CHAPTER 7 · Lists, Keys & Conditional Rendering

### [Easy] Why does React require a key prop on list items?

Keys help React identify which items have changed, been added, or removed. React uses keys during reconciliation to match old virtual DOM nodes with new ones — making updates  $O(n)$  instead of  $O(n^3)$ . Keys must be stable, unique among siblings, and not the array index (unless list never reorders/filters).

```
// ❌ Using index as key — causes issues with reordering
items.map((item, index) => <li key={index}>{item.name}</li>)

// ✅ Using stable unique ID
items.map(item => <li key={item.id}>{item.name}</li>)

// ✅ Full example
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id} className={todo.done ? 'done' : ''}>
          {todo.text}
        </li>
      ))}
    </ul>
  );
}
```

### [Easy] What are the different ways to conditionally render in React?

4 main approaches: (1) if/else before the return, (2) ternary operator inline, (3) && short-circuit operator, (4) switch statement. Choose based on complexity — ternary for simple 2-way, && for optional content, if/else or switch for complex logic.

```
// 1. if/else before return
if (isLoading) return <Spinner />;
if (error) return <Error message={error} />;
return <Data items={items} />;

// 2. Ternary operator
{isLoggedIn ? <UserPanel /> : <LoginButton />}

// 3. && (renders right side only if left is truthy)
// ⚠️ Careful: {0 && <Component />} renders '0'! Use {count > 0 && <C />}
{isAdmin && <AdminBadge />}

// 4. Immediately invoked function / helper
{(() => {
  switch (status) {
    case 'loading': return <Spinner />;
    case 'error': return <Error />;
    default: return <Data />;
  }
})()}()
```

## CHAPTER 8 · Performance Optimization

### [Medium] What is React.memo and how does it work?

React.memo is a Higher-Order Component that memoizes the rendered output of a function component. If props haven't changed (shallow comparison), React skips re-rendering. Use for components that render often with same props. Pass a custom comparison function as second arg for deep comparison.

```
// Without memo - re-renders every time parent re-renders
function ExpensiveChild({ name, value }) {
  return <div>{name}: {value}</div>;
}

// With memo - only re-renders when name or value actually changes
const ExpensiveChild = React.memo(function({ name, value }) {
  return <div>{name}: {value}</div>;
});

// Custom comparison
const MemoChild = React.memo(MyComponent, (prevProps, nextProps) => {
  return prevProps.id === nextProps.id; // return true = skip re-render
});
```

### [Medium] What is code splitting and lazy loading in React?

Code splitting breaks your bundle into smaller chunks loaded on demand. React.lazy() + Suspense lets you dynamically import components. This reduces initial bundle size and improves Time-To-Interactive (TTI).

```
import { lazy, Suspense } from 'react';

// Dynamically import - only loaded when component is needed
const Dashboard = lazy(() => import('./Dashboard'));
const Settings = lazy(() => import('./Settings'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/settings" element={<Settings />} />
      </Routes>
    </Suspense>
  );
}
```

### [Hard] How do you use the React Profiler to find performance issues?

React DevTools Profiler records render timings for each component. Look for: (1) components that render too frequently, (2) renders with long commit time, (3) components where you'd expect no re-render. Also use React.Profiler API programmatically for production monitoring.

```
import { Profiler } from 'react';
```

```
function onRender(id, phase, actualDuration, baseDuration, startTime, commitTime) {
  console.log({ id, phase, actualDuration });
  // phase: 'mount' | 'update'
  // actualDuration: time to render this update
  // baseDuration: estimated time without memoization
}

function App() {
  return (
    <Profiler id="App" onRender={onRender}>
      <MyComponent />
    </Profiler>
  );
}
```

**[Hard] What is windowing / virtualization and when should you use it?**

Windowing renders only visible items in a large list instead of all items. Libraries like react-window or react-virtual implement this. Essential when rendering 1000+ items — without it, the DOM becomes huge and scrolling lags.

```
import { FixedSizeList as List } from 'react-window';

function Row({ index, style }) {
  return <div style={style}>Row {index}</div>;
}

function BigList({ items }) {
  return (
    <List
      height={600} // Visible height
      itemCount={items.length}
      itemSize={50} // Height of each row
      width="100%"
    >
      {Row}
    </List>
  );
}
```

## CHAPTER 9 · Context API & State Management

### [Hard] When should you use Context vs Redux?

Context API: simple global state with infrequent updates (theme, locale, auth user).

Redux/Zustand: complex state with many actions, when you need DevTools time-travel debugging, when many components need same rapidly-updating state. Context re-renders ALL consumers on every value change — avoid putting frequently-changing data in it.

### Custom Context Pattern — Best Practice

```
// auth-context.js
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

// Custom hook — cleaner than calling useContext everywhere
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside AuthProvider');
  return ctx;
}

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = async (credentials) => {
    const data = await loginAPI(credentials);
    setUser(data.user);
  };

  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// Usage anywhere in tree:
function Navbar() {
  const { user, logout } = useAuth();
  return <button onClick={logout}>{user?.name}</button>;
}
```

### Redux Toolkit — Interview Pattern

#### [Medium] Explain the Redux data flow

Action → Reducer → Store → UI. (1) Component dispatches an action object {type, payload}. (2) Reducer receives old state + action, returns new state (pure function, no mutations). (3) Store updates. (4) All subscribed components re-render.

```

// Redux Toolkit (modern Redux)
import { createSlice, configureStore } from '@reduxjs/toolkit';

// Slice = reducer + actions combined
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: state => { state.value += 1; }, // Immer allows 'mutation' syntax
    decrement: state => { state.value -= 1; },
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    },
  },
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;

const store = configureStore({ reducer: { counter: counterSlice.reducer } });

// In component:
function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();
  return (
    <div>
      <button onClick={() => dispatch(decrement())}>-</button>
      <span>{count}</span>
      <button onClick={() => dispatch(increment())}>+</button>
    </div>
  );
}

```

## CHAPTER 10 · React Router

### [Medium] How does React Router work?

React Router is a client-side routing library. It intercepts URL changes (without page reload) and renders the matching component. React Router v6 uses `<Routes>` and `<Route>` elements. Key components: `BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink`, `Navigate`, `Outlet`.

```
import { BrowserRouter, Routes, Route, Link, useParams, useNavigate } from
'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users/:id" element={<UserDetail />} />
        <Route path="/dashboard" element={<ProtectedRoute><Dashboard
/></ProtectedRoute>} />
        <Route path="*" element={<NotFound />} />          {/* 404 */}
      </Routes>
    </BrowserRouter>
  );
}

// Dynamic route - reading params
function UserDetail() {
  const { id } = useParams();
  return <h1>User ID: {id}</h1>;
}

// Programmatic navigation
function LoginPage() {
  const navigate = useNavigate();
  const handleLogin = () => navigate('/dashboard', { replace: true });
  return <button onClick={handleLogin}>Login</button>;
}
```

### [Medium] How do you implement a Protected Route?

A protected route checks if the user is authenticated and redirects to login if not. It wraps the actual component and conditionally renders or redirects.

```
import { Navigate, useLocation } from 'react-router-dom';

function ProtectedRoute({ children }) {
  const { user } = useAuth(); // your auth context
  const location = useLocation();

  if (!user) {
    // Redirect to login, save current path for post-login redirect
    return <Navigate to="/login" state={{ from: location }} replace />;
  }
}
```

```
    }  
    return children;  
  }  
  
  // Usage:  
  <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />  
  
  // After login, redirect back:  
  function LoginPage() {  
    const navigate = useNavigate();  
    const location = useLocation();  
    const from = location.state?.from?.pathname || '/';  
    const handleLogin = async () => {  
      await login(credentials);  
      navigate(from, { replace: true });  
    };  
  }  
}
```

## CHAPTER 11 · Error Boundaries & Suspense

### [Medium] What is an Error Boundary?

Error Boundaries are class components that catch JavaScript errors anywhere in their child component tree and display a fallback UI instead of crashing the whole app. They catch errors during rendering, in lifecycle methods, and in constructors. They do NOT catch errors in event handlers, async code, or SSR.

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true, error }; // Update state to show fallback
  }

  componentDidCatch(error, errorInfo) {
    // Log to error reporting service (Sentry, etc.)
    logErrorToService(error, errorInfo.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback || <h2>Something went wrong.</h2>;
    }
    return this.props.children;
  }
}

// Usage
<ErrorBoundary fallback={<ErrorPage />}>
  <MyWidget />
</ErrorBoundary>
```



## CHAPTER 12 · Higher-Order Components & Render Props

### [Medium] What is a Higher-Order Component (HOC)?

A HOC is a function that takes a component and returns a new enhanced component. It's a pattern for reusing component logic. Convention: prefix with 'with' (withAuth, withLogging). Note: hooks have largely replaced HOCs in modern React.

```
// HOC that adds authentication check
function withAuth(WrappedComponent) {
  return function AuthenticatedComponent(props) {
    const { user } = useAuth();
    if (!user) return <Navigate to="/login" />;
    return <WrappedComponent {...props} user={user} />;
  };
}

// HOC that adds loading state
function withLoading(WrappedComponent) {
  return function({ isLoading, ...props }) {
    if (isLoading) return <Spinner />;
    return <WrappedComponent {...props} />;
  };
}

// Usage
const ProtectedDashboard = withAuth(Dashboard);
const DashboardWithLoading = withLoading(ProtectedDashboard);
```

### [Medium] What is the Render Props pattern?

Render Props is a pattern where a component receives a function prop that it calls to render something. It allows sharing stateful logic between components. The children prop or any prop can be the 'render prop'.

```
// Mouse tracker with render prop
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  const handleMouseMove = (e) => {
    setPosition({ x: e.clientX, y: e.clientY });
  };

  return (
    <div onMouseMove={handleMouseMove} style={{ height: '100vh' }}>
      {render(position)}
    </div>
  );
}

// Usage
<MouseTracker render={({ x, y }) => (
  <p>Mouse is at ({x}, {y})</p>
)} />
```

## CHAPTER 13 · Custom Hooks

### [Medium] What is a custom hook? How do you create one?

A custom hook is a JavaScript function whose name starts with 'use' that calls other hooks. It extracts reusable stateful logic out of components. Each call to a custom hook gets its own isolated state.

### Must-Know Custom Hooks for Interviews

```
// 1. useFetch - generic data fetching hook
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    fetch(url)
      .then(r => r.json())
      .then(d => { if (!cancelled) { setData(d); setLoading(false); } })
      .catch(e => { if (!cancelled) { setError(e); setLoading(false); } });
    return () => { cancelled = true; };
  }, [url]);

  return { data, loading, error };
}

// 2. useLocalStorage - persisted state
function useLocalStorage(key, initialValue) {
  const [stored, setStored] = useState(() => {
    try {
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch { return initialValue; }
  });

  const setValue = (value) => {
    const v = value instanceof Function ? value(stored) : value;
    setStored(v);
    localStorage.setItem(key, JSON.stringify(v));
  };

  return [stored, setValue];
}

// 3. useDebounce - delay expensive operations
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
  return debounced;
}

// 4. usePrevious - track previous value
```

```
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; });
  return ref.current;
}

// 5. useClickOutside - detect clicks outside an element
function useClickOutside(ref, handler) {
  useEffect(() => {
    const listener = (e) => {
      if (!ref.current || ref.current.contains(e.target)) return;
      handler(e);
    };
    document.addEventListener('mousedown', listener);
    return () => document.removeEventListener('mousedown', listener);
  }, [ref, handler]);
}
```

## CHAPTER 14 · Testing in React

### [Medium] What tools do you use to test React components?

Jest (test runner + assertions), React Testing Library (RTL — testing from user's perspective, queries DOM like a user would), and optionally Cypress (end-to-end). RTL philosophy: test behavior, not implementation details.

```
// Basic RTL test
import { render, screen, fireEvent } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

// Component under test
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p data-testid="count">Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
    </div>
  );
}

// Test
describe('Counter', () => {
  it('starts at 0', () => {
    render(<Counter />);
    expect(screen.getByTestId('count')).toHaveTextContent('Count: 0');
  });

  it('increments on button click', async () => {
    const user = userEvent.setup();
    render(<Counter />);
    await user.click(screen.getByRole('button', { name: 'Increment' }));
    expect(screen.getByTestId('count')).toHaveTextContent('Count: 1');
  });
});
```

### [Hard] How do you test custom hooks?

Use `renderHook` from `@testing-library/react`. It renders the hook in a test component so you can assert on its return values and trigger updates.

```
import { renderHook, act } from '@testing-library/react';
import { useCounter } from './useCounter';

test('useCounter increments correctly', () => {
  const { result } = renderHook(() => useCounter(0));

  expect(result.current.count).toBe(0);

  act(() => {
    result.current.increment();
  });

  expect(result.current.count).toBe(1);
});
```



## CHAPTER 15 · Live Coding Challenges

### Interview Tip

These are actual tasks asked in interviews. Practice writing them from scratch in 10-15 minutes each. Narrate your thought process while coding.

### Challenge 1: Todo App (Most Common Interview Task)

Build a todo list with: add, delete, toggle complete, and filter (all/active/completed).

```
import { useState } from 'react';

function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');
  const [filter, setFilter] = useState('all');

  const addTodo = () => {
    if (!input.trim()) return;
    setTodos(prev => [...prev, { id: Date.now(), text: input, done: false }]);
    setInput('');
  };

  const toggleTodo = (id) =>
    setTodos(prev => prev.map(t => t.id === id ? { ...t, done: !t.done } : t));

  const deleteTodo = (id) =>
    setTodos(prev => prev.filter(t => t.id !== id));

  const filtered = todos.filter(t => {
    if (filter === 'active') return !t.done;
    if (filter === 'completed') return t.done;
    return true;
  });

  return (
    <div>
      <h1>Todos</h1>
      <div>
        <input value={input} onChange={e => setInput(e.target.value)}
          onKeyDown={e => e.key === 'Enter' && addTodo()}
          placeholder="What needs to be done?" />
        <button onClick={addTodo}>Add</button>
      </div>

      <div>
        {[ 'all', 'active', 'completed' ].map(f => (
          <button key={f} onClick={() => setFilter(f)}
            style={{ fontWeight: filter === f ? 'bold' : 'normal' }}>
            {f}
          </button>
        ))}
      </div>

      <ul>
        {filtered.map(todo => (
```

```

        <li key={todo.id} style={{ textDecoration: todo.done ? 'line-through' :
'none' }}>
          <input type="checkbox" checked={todo.done} onChange={() =>
toggleTodo(todo.id)} />
          {todo.text}
          <button onClick={() => deleteTodo(todo.id)}>X</button>
        </li>
      )}
    </ul>
    <p>{todos.filter(t => !t.done).length} items left</p>
  </div>
);
}

```

## Challenge 2: Infinite Scroll

```

import { useState, useEffect, useRef, useCallback } from 'react';

function InfiniteScroll() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [loading, setLoading] = useState(false);
  const [hasMore, setHasMore] = useState(true);
  const observerRef = useRef(null);

  // Fetch page of data
  useEffect(() => {
    if (!hasMore) return;
    setLoading(true);
    fetch(`https://jsonplaceholder.typicode.com/posts?_page=${page}&_limit=10`)
      .then(r => r.json())
      .then(data => {
        setItems(prev => [...prev, ...data]);
        setHasMore(data.length === 10);
        setLoading(false);
      });
  }, [page]);

  // Intersection Observer - fires when last element is visible
  const lastItemRef = useCallback((node) => {
    if (loading) return;
    if (observerRef.current) observerRef.current.disconnect();
    observerRef.current = new IntersectionObserver(entries => {
      if (entries[0].isIntersecting && hasMore) {
        setPage(prev => prev + 1);
      }
    });
  }, []);

  if (node) observerRef.current.observe(node);
}, [loading, hasMore]);

return (
  <ul>
    {items.map((item, i) => (
      <li
        key={item.id}
        ref={i === items.length - 1 ? lastItemRef : null}
      >
        {item.title}
      </li>
    ))}
  </ul>
)

```

```

    {loading && <li>Loading...</li>}
  </ul>
);
}

```

### Challenge 3: Debounced Search with API

```

function SearchComponent() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);

  // Debounce: wait until user stops typing for 500ms
  const debouncedQuery = useDebounce(query, 500); // our custom hook from Ch.13

  useEffect(() => {
    if (!debouncedQuery.trim()) { setResults([]); return; }

    setLoading(true);
    fetch(`https://api.github.com/search/users?q=${debouncedQuery}`)
      .then(r => r.json())
      .then(data => {
        setResults(data.items || []);
        setLoading(false);
      });
  }, [debouncedQuery]);

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search GitHub users..."
      />
      {loading && <p>Searching...</p>}
      <ul>
        {results.map(user => (
          <li key={user.id}>
            <img src={user.avatar_url} alt={user.login} width={30} />
            {user.login}
          </li>
        ))}
      </ul>
    </div>
  );
}

```

### Challenge 4: Accordion Component

```

function Accordion({ items }) {
  const [openIndex, setOpenIndex] = useState(null);

  const toggle = (index) => setOpenIndex(prev => prev === index ? null : index);

  return (
    <div>
      {items.map((item, i) => (
        <div key={i} style={{ border: '1px solid #ddd', marginBottom: 4 }}>

```



```

        <button
          onClick={() => toggle(i)}
          style={{ width: '100%', textAlign: 'left', padding: 12, background:
'#f5f5f5' }}
        >
          {item.title}
          <span style={{ float: 'right' }}>{openIndex === i ? '▲' : '▼'}</span>
        </button>
        {openIndex === i && (
          <div style={{ padding: 12 }}>{item.content}</div>
        )}
      </div>
    )}
  </div>
);
}

```

## Challenge 5: Drag & Drop List (no library)

```

function DragList() {
  const [items, setItems] = useState(['Item A', 'Item B', 'Item C', 'Item D']);
  const dragIndex = useRef(null);

  const handleDragStart = (i) => { dragIndex.current = i; };

  const handleDrop = (i) => {
    const updated = [...items];
    const [dragged] = updated.splice(dragIndex.current, 1);
    updated.splice(i, 0, dragged);
    setItems(updated);
    dragIndex.current = null;
  };

  return (
    <ul style={{ listStyle: 'none', padding: 0 }}>
      {items.map((item, i) => (
        <li
          key={item}
          draggable
          onDragStart={() => handleDragStart(i)}
          onDragOver={(e) => e.preventDefault()}
          onDrop={() => handleDrop(i)}
          style={{ padding: 12, margin: 4, background: '#e3f2fd', cursor: 'grab',
borderRadius: 4 }}
        >
          ≡ {item}
        </li>
      ))}
    </ul>
  );
}

```

## CHAPTER 16 · Advanced & Expert-Level Questions

### [Expert] What is Concurrent Mode / Concurrent React?

Concurrent React (React 18+) allows React to work on multiple state updates simultaneously and interrupt low-priority renders for high-priority ones. Enabled by `createRoot()`. Key APIs: `useTransition` (mark an update as low priority), `useDeferredValue` (defer re-render of slow parts), `startTransition`.

```
import { useTransition, useDeferredValue } from 'react';

// useTransition - mark heavy update as non-urgent
function FilterList() {
  const [query, setQuery] = useState('');
  const [isPending, startTransition] = useTransition();
  const [filteredItems, setFiltered] = useState(allItems);

  const handleChange = (e) => {
    setQuery(e.target.value); // Urgent - update input immediately
    startTransition(() => {
      setFiltered(filterItems(e.target.value)); // Non-urgent - can be deferred
    });
  };

  return (
    <>
      <input value={query} onChange={handleChange} />
      {isPending && <span>Loading...</span>}
      <List items={filteredItems} />
    </>
  );
}

// useDeferredValue - defer re-rendering based on a value
function App({ text }) {
  const deferredText = useDeferredValue(text);
  // Expensive component only re-renders with deferred (delayed) value
  return <HeavyList filter={deferredText} />;
}
```

### [Expert] What are React Server Components (RSC)?

RSC (React 18+/Next.js 13+) are components that render on the server and send HTML to the client — no JS sent to browser for these components. They can directly access databases, file system, backend APIs. Client components (marked 'use client') handle interactivity. RSC reduces bundle size and improves performance.

### [Expert] What is the `useImperativeHandle` hook?

`useImperativeHandle` customizes what ref value is exposed to the parent when using `forwardRef`. Instead of exposing the raw DOM node, you can expose specific methods. Use sparingly — React's declarative model is almost always better.

```
const FancyInput = forwardRef((props, ref) => {
  const inputRef = useRef();
```

```

    useImperativeHandle(ref, () => ({
      focus: () => inputRef.current.focus(),
      clear: () => { inputRef.current.value = ''; },
    }));

    return <input ref={inputRef} {...props} />;
  });

// Parent usage:
function Parent() {
  const ref = useRef();
  return (
    <>
      <FancyInput ref={ref} />
      <button onClick={() => ref.current.clear()}>Clear</button>
    </>
  );
}

```

### [Expert] Explain the Stale Closure problem in React hooks

A stale closure happens when a hook (usually `useEffect` or `useCallback`) captures a variable from a previous render and uses the old value. Fix: add the variable to the dependency array, or use a `ref` to store the latest value.

```

// ❌ Stale closure - timer always logs count = 0
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const timer = setInterval(() => {
      console.log(count); // Captures count=0, never updates!
    }, 1000);
    return () => clearInterval(timer);
  }, []); // Missing 'count' dependency
}

// ✅ Fix 1: Add count to deps (but resets timer on each change)
useEffect(() => {
  const timer = setInterval(() => console.log(count), 1000);
  return () => clearInterval(timer);
}, [count]);

// ✅ Fix 2: Use a ref to always have latest value
const countRef = useRef(count);
useEffect(() => { countRef.current = count; });

useEffect(() => {
  const timer = setInterval(() => console.log(countRef.current), 1000);
  return () => clearInterval(timer);
}, []);

```

### [Expert] How does React handle batching and what changed in React 18?

Before React 18, batching only happened in React event handlers. Async code (`setTimeout`, Promises, native events) triggered individual re-renders for each `setState` call. React 18 introduced Automatic Batching — all updates everywhere are batched by default. Use `flushSync()` to opt out of batching for critical updates.

```
// React 18 — all these setState calls are batched into ONE render
setTimeout(() => {
  setA(1);    // Previously: caused re-render
  setB(2);    // Previously: caused re-render
  setC(3);    // Previously: caused re-render
              // Now: ONE re-render after all three
}, 1000);

// Opt out with flushSync
import { flushSync } from 'react-dom';
flushSync(() => setA(1));    // Forces immediate re-render
flushSync(() => setB(2));    // Forces immediate re-render
```

**[Hard] What is forwardRef and when do you need it?**

By default, ref on a component doesn't work — refs don't get passed as props. forwardRef wraps a component to accept a ref from its parent and forward it to an inner DOM element or component.

## QUICK REFERENCE — CHEAT SHEET

### Hooks at a Glance

|                                    |                                                                    |
|------------------------------------|--------------------------------------------------------------------|
| <code>useState(init)</code>        | Local state. Returns [value, setter].                              |
| <code>useEffect(fn, deps)</code>   | Side effects. Runs after render. Cleanup with return fn.           |
| <code>useContext(Ctx)</code>       | Read context value. Triggers re-render on value change.            |
| <code>useRef(init)</code>          | Mutable ref, no re-render on change. Also for DOM refs.            |
| <code>useMemo(fn, deps)</code>     | Memoize expensive computed value.                                  |
| <code>useCallback(fn, deps)</code> | Memoize function reference. Prevent child re-renders.              |
| <code>useReducer(fn, init)</code>  | Complex state logic. Alternative to useState.                      |
| <code>useLayoutEffect(fn)</code>   | Like useEffect but fires synchronously after DOM mutation.         |
| <code>useId()</code>               | Generate stable unique IDs for accessibility.                      |
| <code>useTransition()</code>       | Mark update as low-priority. Returns [isPending, startTransition]. |
| <code>useDeferredValue(v)</code>   | Defer re-render of expensive derived state.                        |
| <code>useImperativeHandle</code>   | Customize exposed ref API. Used with forwardRef.                   |

### Performance Checklist

- Wrap pure components with `React.memo` to prevent unnecessary re-renders
- Use `useCallback` for functions passed as props to memoized children
- Use `useMemo` for expensive computations (filter, sort, complex transforms)
- Code-split routes with `React.lazy()` + `Suspense`
- Virtualize long lists with `react-window` or `react-virtual`
- Move expensive context values to a separate context to narrow re-renders
- Use the key prop correctly — never use array index for dynamic/sortable lists
- Profile with React DevTools Profiler before optimizing

## Common Interview Traps

- Forgetting to add deps to `useEffect` → stale closure / missing update
- Using `0 && <Component>` — renders the number 0. Use `!!0` or `count > 0`
- Mutating state directly (`array.push`, `obj.prop = v`) — no re-render
- Calling async function directly as `useEffect` callback
- Using index as key when list items can be reordered or filtered
- Overusing `useMemo/useCallback` — premature optimization
- Not cleaning up `useEffect` (timer, subscription, `AbortController`) → memory leak
- `setState` inside render — causes infinite loop

**Good Luck in Your Interview!** 

Remember: Understand the WHY behind every answer, not just the WHAT.

Practice coding challenges daily • Stay curious • Build real projects